

PROJECT REPORT

Movie Reservation System

A Modular Console-Based Ticket Booking Application in Python

Item	Details
Project Title	Movie Reservation System
Programming Language	Python 3
Interface	Command Line (Console)
Modules	main.py, movies.py, booking.py, display.py
Currency	Indian Rupee (INR)
Document Type	Project Report

Submitted by: Aminesh Shrivastava

Registration No. : 26 BCE11537

Acknowledgement

I take this opportunity to express my sincere gratitude to everyone who helped me in completing this project. I am thankful to my teachers and project guide for their constant encouragement, valuable suggestions and patient guidance throughout the development of the Movie Reservation System.

I also thank my family and friends for their support and motivation. Their feedback during testing helped me find and fix several issues in the early versions of the program. Finally, I acknowledge the vast Python community, whose documentation, tutorials and open discussions made learning the language an enjoyable experience.

Abstract

The **Movie Reservation System** is a console-based application written in Python that allows a user to view the list of movies currently on show, reserve tickets for a chosen movie and review all the bookings made during the running session. Every movie is stored with a unique serial number, a title and a ticket price in Indian Rupees. When a user reserves tickets, the system multiplies the ticket price by the number of tickets, records the booking as a dictionary, and displays a confirmation message with the total cost.

The application is divided into four modules to keep each responsibility separate. *movies.py* holds the movie catalogue, *booking.py* contains the reservation logic and the booking history, *display.py* renders the main menu, and *main.py* acts as the entry point and controls the flow of the program through an infinite loop that ends when the user chooses to exit.

This report documents the problem statement, requirements, design, implementation, testing and limitations of the system. It also includes a critical review of the current code, identifying input-validation gaps and design improvements, and it outlines a roadmap for future work such as persistent storage, seat selection, a graphical user interface and online payments.

Keywords

Python, Movie Ticket Booking, Console Application, Modular Programming, Dictionary, Reservation System.

Table of Contents

Certificate	2
Acknowledgement	3
Abstract	4
Table of Contents	5
1. Introduction	8
1.1 Background	8
1.2 Motivation	8
1.3 Objectives	8
1.4 Scope of the Project	8
1.5 Organisation of the Report	9
2. Problem Statement and Existing System	10
2.1 Problem Statement	10
2.2 Existing System	10
2.3 Proposed System	10
2.4 Advantages of the Proposed System	10
3. Requirement Analysis	11
3.1 Functional Requirements	11
3.2 Non-Functional Requirements	11
3.3 Hardware Requirements	11
3.4 Software Requirements	12
3.5 Feasibility Study	12
4. System Design	13
4.1 Design Approach	13
4.2 Architecture	13
4.3 Data Design	13
4.4 Menu Design	14
4.5 Flowchart of the Main Loop	14
4.6 Flowchart of the Booking Process	15
4.7 Data Flow Overview	16
5. Implementation	17

5.1 movies.py - Movie Catalogue	17
5.2 display.py - Menu Presentation	17
5.3 booking.py - Reservation Logic	17
5.4 main.py - Program Controller	18
5.5 Python Concepts Demonstrated	19
5.6 Module Interaction	19
6. Execution Walkthrough and Sample Output	21
6.1 Running the Program	21
6.2 Scenario 1: Displaying Movies	21
6.3 Scenario 2: Booking Tickets	21
6.4 Scenario 3: Viewing Bookings	21
6.5 Scenario 4: Invalid Movie Number	21
6.6 Scenario 5: Exiting	21
6.7 Worked Price Calculations	22
7. Testing	23
7.1 Testing Strategy	23
7.2 Test Cases	23
7.3 Test Summary	23
8. Code Review: Limitations and Issues Identified	25
8.1 Missing Input Validation	25
8.2 Non-Positive Ticket Counts	25
8.3 Duplicate display_movies Functions	25
8.4 Inconsistent Price Formatting	25
8.5 Data Is Not Persistent	25
8.6 Global State	25
8.7 Currency Symbol and Encoding	25
8.8 Other Observations	26
9. Recommended Fixes and Improved Code	27
9.1 Safe Integer Input Helper	27
9.2 Improved book_ticket()	27
9.3 Single display_movies Function	27
9.4 Safe Main Loop	27

9.5 Saving Bookings to a File	28
9.6 Expected Effect on Test Results	28
10. Future Enhancements	29
10.1 Short-Term Improvements	29
10.2 Medium-Term Improvements	29
10.3 Long-Term Improvements	29
10.4 Suggested Class Design	29
11. Conclusion	31
12. References	32
Appendix A: Complete Source Code	33
A.1 main.py	33
A.2 movies.py	33
A.3 booking.py	33
A.4 display.py	34
Appendix B: Glossary	35
Appendix C: Installation and Usage Guide	36
C.1 Installing Python	36
C.2 Preparing the Project	36
C.3 Running	36
C.4 Customising the Catalogue	36
C.5 Troubleshooting	36

1. Introduction

1.1 Background

Cinema remains one of the most popular forms of entertainment in India and around the world. Historically, moviegoers had to stand in long queues at the theatre counter to buy tickets, often risking a sold-out show. With the growth of computing and the internet, ticket booking has moved from paper registers to software systems that can display shows, calculate prices and record reservations in seconds.

Reservation systems are also a classic teaching example in computer science. They combine data storage, user interaction, arithmetic, decision making and reporting in one compact problem, which makes them ideal for learners who are practising programming fundamentals. The Movie Reservation System presented in this report is such a project: small enough to understand completely, yet realistic enough to illustrate how larger booking platforms work.

1.2 Motivation

The motivation for this project was to apply the concepts learned in an introductory Python course to a real-world scenario. Instead of writing isolated exercises on loops or dictionaries, building a complete application forces the developer to think about how different parts of a program communicate, how data flows between functions and how a user will actually interact with the software.

1.3 Objectives

- To design a simple menu-driven application for reserving movie tickets.
- To store the movie catalogue in a suitable Python data structure with serial number, title and price.
- To allow a user to select a movie, enter the number of tickets and receive an instant price calculation.
- To maintain a list of bookings made in the current session and display them on request.
- To organise the code into separate modules following the principle of separation of concerns.
- To document, test and critically evaluate the implementation.

1.4 Scope of the Project

The system is intended for a single user working on a single computer through a text terminal. It covers listing of movies, booking of tickets, and viewing of booking history. It

does not cover seat maps, show timings, multiple theatres, user accounts, payment gateways or persistent storage. These items are treated as future enhancements in Chapter 10.

1.5 Organisation of the Report

Chapter	Content
1	Introduction, motivation, objectives and scope
2	Problem statement and existing system study
3	Requirement analysis and feasibility
4	System design: architecture, data design and flow
5	Implementation of every module with explanation
6	Execution walkthrough and sample output
7	Testing strategy and test cases
8	Code review: limitations and issues identified
9	Recommended fixes and improved code
10	Future enhancements
11	Conclusion, references and appendices

Table 1.1: Structure of this report

2. Problem Statement and Existing System

2.1 Problem Statement

Manual ticket booking at a cinema counter is slow, error prone and difficult to audit. Clerks must recall prices, calculate totals by hand and write records in a register. Customers cannot easily see which movies are available without asking, and there is no quick way to look up earlier bookings.

The problem addressed in this project is: **to develop a software application that presents the list of available movies with their prices, lets a user reserve any number of tickets for a selected movie, computes the total amount automatically, and lets the user review all reservations made.**

2.2 Existing System

In the manual system, information about movies and prices is displayed on a notice board or poster. A booking clerk issues a printed ticket and notes the sale in a ledger. The main drawbacks are listed below.

- Calculation of totals is done by hand and can be wrong.
- Records are on paper and are hard to search or summarise.
- The customer has no single place to review all reservations.
- Adding or removing a movie requires reprinting notices and retraining staff.

2.3 Proposed System

The proposed Movie Reservation System replaces the manual process with a menu-driven Python program. The catalogue is held in a dictionary so that movies can be looked up instantly by serial number. Bookings are stored in a list of dictionaries, so each reservation carries its movie name, number of tickets and total price. The program guides the user through each step with clear prompts and messages.

2.4 Advantages of the Proposed System

- **Speed:** a booking is completed in a few keystrokes.
- **Accuracy:** the total price is calculated by the program.
- **Transparency:** the user sees a full confirmation summary.
- **Maintainability:** the catalogue can be changed by editing one dictionary.
- **Modularity:** each file has one clearly defined purpose.

3. Requirement Analysis

3.1 Functional Requirements

ID	Requirement	Priority
FR1	The system shall display a numbered main menu with four options.	High
FR2	The system shall display all available movies with number, title and price.	High
FR3	The system shall let the user choose a movie by its number.	High
FR4	The system shall accept the number of tickets and compute total = tickets x price.	High
FR5	The system shall store each successful booking in memory.	High
FR6	The system shall display all stored bookings, or a message if none exist.	Medium
FR7	The system shall reject a movie number that is not in the catalogue.	High
FR8	The system shall return to the menu after every action until the user exits.	High
FR9	The system shall terminate with a thank-you message when the user selects Exit.	Medium

Table 3.1: Functional requirements

3.2 Non-Functional Requirements

- **Usability:** prompts and menus must be simple and self-explanatory.
- **Portability:** the program should run on Windows, Linux and macOS with any Python 3 interpreter.
- **Maintainability:** code must be split into small, single-purpose modules.
- **Performance:** every operation should respond instantly for a catalogue of this size.
- **Reliability:** the program should not crash on normal, valid usage.

3.3 Hardware Requirements

Component	Minimum Requirement
Processor	Any dual-core processor, 1 GHz or above
RAM	512 MB
Storage	10 MB free disk space
Display	Any terminal or console window
Input	Standard keyboard

Table 3.2: Hardware requirements

3.4 Software Requirements

Component	Requirement
Operating System	Windows 10/11, Linux or macOS
Language	Python 3.8 or later
Editor / IDE	IDLE, VS Code, PyCharm or any text editor
External libraries	None (standard library only)

Table 3.3: Software requirements

3.5 Feasibility Study

Technical Feasibility

The project uses only core Python features and needs no third-party packages. Python is freely available and runs on all common operating systems, so the project is technically feasible.

Economic Feasibility

All tools used are free and open source. The program needs no server or paid service, so the development and running cost is effectively zero.

Operational Feasibility

The console interface consists of numbered menus that anyone able to type a number can use. No special training is required, which makes the system operationally feasible.

4. System Design

4.1 Design Approach

The design follows a modular, procedural approach. The program is decomposed into four files, each responsible for one concern: data, business logic, presentation of the menu, and program control. This decomposition makes each file short, easy to read and easy to test independently.

4.2 Architecture

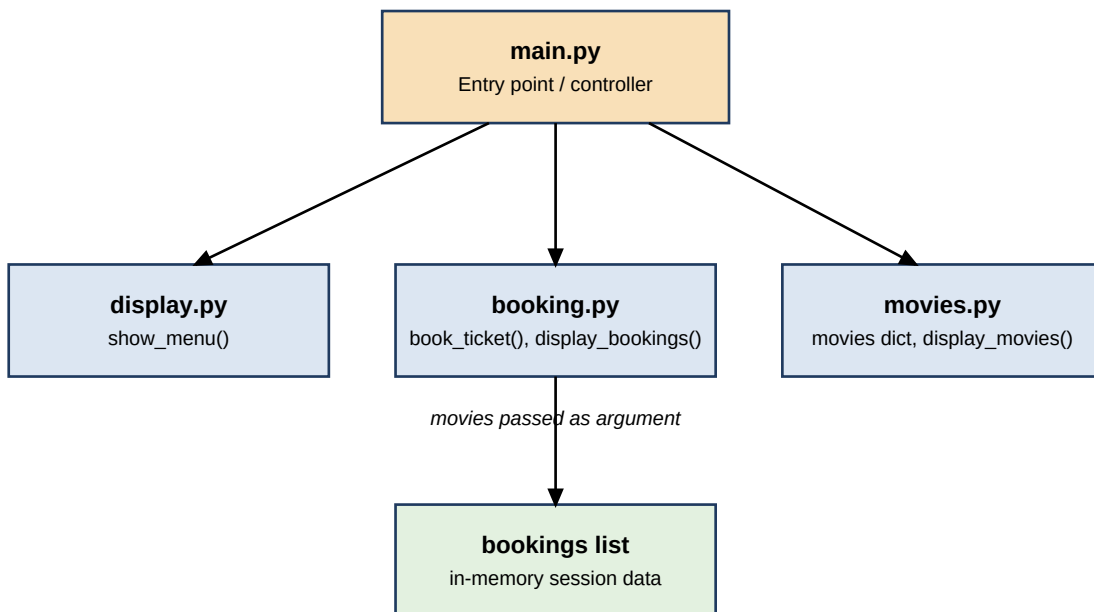


Figure 4.1: Module architecture of the Movie Reservation System

main.py imports functions and data from the other three modules and drives the user interaction. **display.py** only prints the menu. **movies.py** owns the catalogue. **booking.py** owns the booking list and the reservation logic; it receives the catalogue as a function argument rather than importing it, which keeps the dependency direction clean.

4.3 Data Design

4.3.1 Movie Catalogue

The catalogue is a Python dictionary. The key is an integer serial number and the value is a tuple (`title`, `price`). A dictionary gives constant-time lookup by number, and a tuple is appropriate because a movie record is not modified after creation.

Key (int)	Title (str)	Price in INR (int)
1	Ramayana	200
2	Godzilla Minus Zero	180

Key (int)	Title (str)	Price in INR (int)
3	Varanasi	200
4	Spirit	200
5	The Angry Birds Movie 3	180

Table 4.1: Contents of the movies dictionary

4.3.2 Booking Record

Each successful reservation is stored as a dictionary with three fields and appended to the global list bookings.

Field	Type	Description
movie	str	Title of the booked movie
tickets	int	Number of tickets booked
total	int	tickets multiplied by unit price

Table 4.2: Structure of a booking record

4.4 Menu Design

```
-----Movie Reservation System-----  
1. Display Available Movies  
2. Reserve a Movie Ticket  
3. Display Your Bookings  
4. Exit  
Enter your choice:
```

4.5 Flowchart of the Main Loop

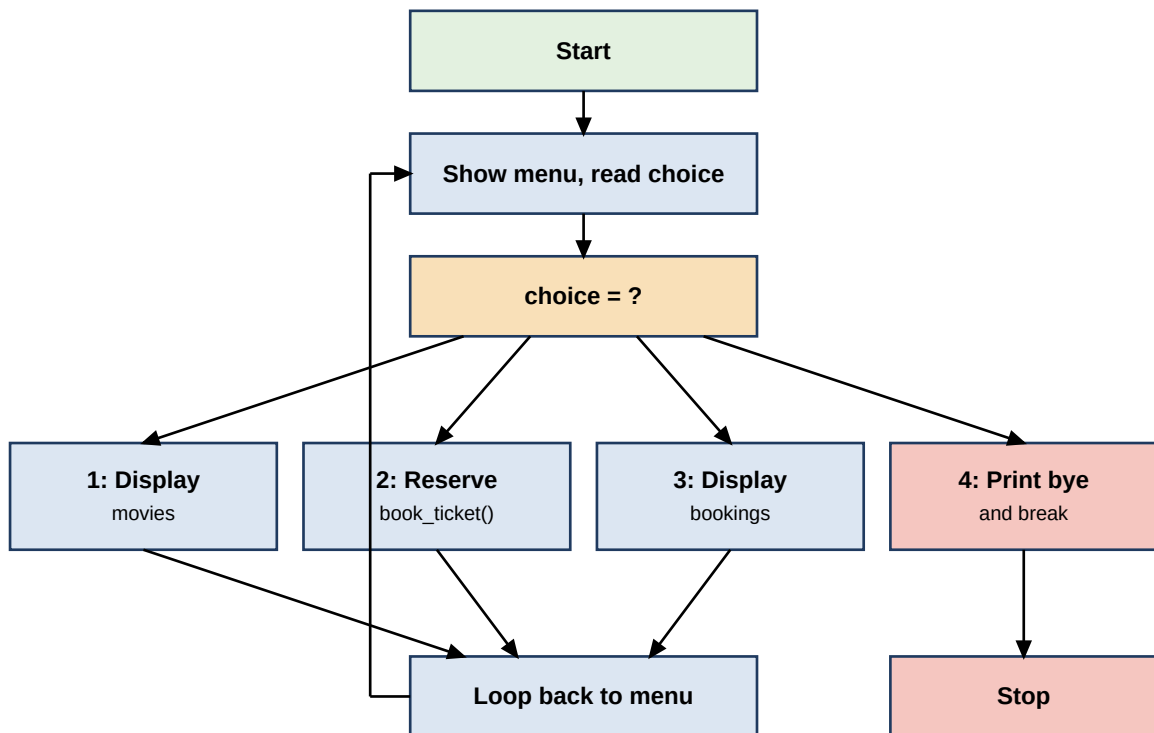


Figure 4.2: Flowchart of the main program loop

4.6 Flowchart of the Booking Process

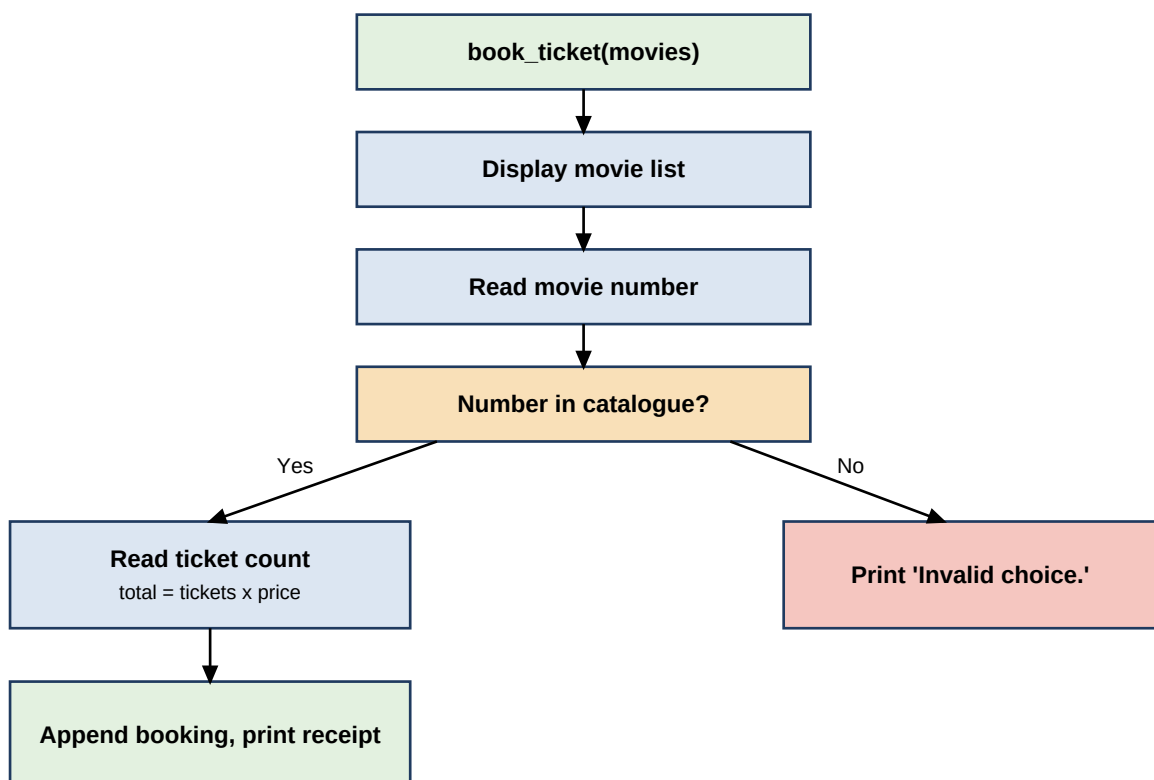


Figure 4.3: Flowchart of ticket booking

4.7 Data Flow Overview

The user's keyboard input enters the system through `input()`. The choice is read by the main loop and dispatched to the appropriate function. During booking, the movie number is validated against the dictionary keys, the ticket count is multiplied by the price found in the tuple, and the resulting record is appended to the bookings list. The list is later read by `display_bookings()` and formatted for the screen. No data leaves the program's memory; when the program ends, all bookings are lost.

5. Implementation

This chapter explains every source file in the project. The complete listings are also reproduced in Appendix A.

5.1 movies.py - Movie Catalogue

```
movies = {
    1: ("Ramayana", 200),
    2: ("Godzilla Minus Zero",180),
    3: ("Varanasi", 200),
    4: ("Spirit",200),
    5: ("The Angry Birds Movie 3",180)
}

def display_movies():
    print("\n-----Available Movies-----")
    for number, details in movies.items():
        print(f"{number}. {details[0]} - {details[1]}")
```

The module defines the global dictionary `movies` and a function `display_movies()` that prints it. The loop uses `movies.items()`, which yields each key with its value. The tuple is indexed with `details[0]` for the title and `details[1]` for the price. An f-string builds each output line. Adding a new movie only requires a new key-value pair in the dictionary; no other code needs to change.

5.2 display.py - Menu Presentation

```
def show_menu():
    print("-----Movie Reservation System-----")
    print("1. Display Available Movies")
    print("2. Reserve a Movie Ticket")
    print("3. Display Your Bookings")
    print("4. Exit")
```

This is the simplest module. It holds a single function that prints the four menu options. Keeping the menu in its own file means that its wording or layout can be modified without touching the control logic.

5.3 booking.py - Reservation Logic

5.3.1 The bookings list

```
bookings = []
```

A module-level empty list stores every booking made during the session. Because the list lives at module level, both `book_ticket()` and `display_bookings()` can access it. Appending to a list does not require the `global` keyword because the list object is mutated, not reassigned.

5.3.2 book_ticket()

```
def book_ticket(movies):
```

```
display_movies(movies)

choice = int(input("Enter the number of the movie you want to book: "))
if choice in movies:
    movie_name, price = movies[choice]
    tickets = int(input(f"Enter the number of tickets you want to book for {movie_name}: "))
    total = tickets * price
    booking = {
        "movie": movie_name,
        "tickets": tickets,
        "total": total
    }
    bookings.append(booking)
    print("Booking successful!")
    print("Booking Details:")
    print(f"Movie: {movie_name}")
    print(f"Tickets: {tickets}")
    print(f"Total Price: Rs.{total}")
else:
    print("Invalid choice.")
```

The function takes the catalogue as a parameter, shows the list, and reads the movie number. The membership test `choice in movies` checks the dictionary keys. If the number is valid, tuple unpacking assigns the title and price to `movie_name` and `price`. The user is then asked for the number of tickets, the total is computed, and a dictionary is appended to the list. A confirmation with all details is printed. If the number is not a key, the function prints an error and returns to the menu.

5.3.3 display_bookings()

```
def display_bookings():
    print("-----Your Bookings-----")
    if len(bookings) == 0:
        print("No bookings found.")
    else:
        for i in range(len(bookings)):
            print("Booking", i + 1)
            print("Movie:", bookings[i]["movie"])
            print("Tickets:", bookings[i]["tickets"])
            print("Total Price: Rs.", bookings[i]["total"])
```

If the list is empty, a friendly message is shown. Otherwise the function iterates by index and prints each record, numbering bookings from 1 for readability.

5.3.4 display_movies(movies)

```
def display_movies(movies):
    print("\n-----Available Movies-----")
    for number, details in movies.items():
        print(f"{number}. {details[0]} - {details[1]}")
```

A second version of the listing function exists in `booking.py`. It accepts the catalogue as an argument, which makes it usable independently of the global variable in `movies.py`. It is called at the start of `book_ticket()`.

5.4 main.py - Program Controller

```
from movies import movies, display_movies
from booking import book_ticket, display_bookings
from display import show_menu
while True:
    show_menu()
    choice = int(input("Enter your choice: "))
    if (choice == 1):
        display_movies()
    elif (choice == 2):
        book_ticket(movies)
    elif (choice == 3):
        display_bookings()
    elif (choice == 4):
        print("Exiting the program. Thank you for using the Movie Reservation System!")
        break
    else:
        print("Invalid choice. Please try again.")
```

The three import lines pull in the catalogue, the booking functions and the menu. The `while True` loop shows the menu repeatedly, reads the user's choice and uses an `if / elif / else` ladder to dispatch to the right function. Option 4 prints a farewell message and executes `break`, which ends the loop and the program. Any other number triggers the error message and the menu is displayed again.

5.5 Python Concepts Demonstrated

Concept	Where it is used
Dictionary	movies catalogue keyed by serial number; booking records
Tuple and unpacking	(title, price) values; <code>movie_name, price = movies[choice]</code>
List	bookings list storing all reservations
Functions and parameters	<code>show_menu()</code> , <code>book_ticket(movies)</code> , <code>display_bookings()</code>
Modules and imports	<code>from movies import ...</code> , <code>from booking import ...</code>
Loops	<code>while True</code> menu loop; for loops for listing
Conditionals	<code>if / elif / else</code> dispatch and validation
f-strings	Formatted output of titles, prices and totals
Type conversion	<code>int(input(...))</code> to read numeric input

Table 5.1: Python concepts used in the project

5.6 Module Interaction

When the program starts, Python executes *main.py*. The import statements load *movies.py*, *booking.py* and *display.py* and execute their top-level code once: the catalogue dictionary is created and the empty `bookings` list is initialised. From that point on, the objects are shared for the entire run. Because Python caches imported modules, the `bookings` list is the same object every time `book_ticket()` or `display_bookings()` is called, which is

why bookings accumulate across menu iterations.

6. Execution Walkthrough and Sample Output

6.1 Running the Program

Place all four files in the same folder, open a terminal in that folder and run the entry point:

```
python main.py
```

6.2 Scenario 1: Displaying Movies

```
-----Movie Reservation System-----
1. Display Available Movies
2. Reserve a Movie Ticket
3. Display Your Bookings
4. Exit
Enter your choice: 1

-----Available Movies-----
1. Ramayana - 200
2. Godzilla Minus Zero - 180
3. Varanasi - 200
4. Spirit - 200
5. The Angry Birds Movie 3 - 180
```

6.3 Scenario 2: Booking Tickets

```
Enter your choice: 2

-----Available Movies-----
1. Ramayana - 200
...
Enter the number of the movie you want to book: 2
Enter the number of tickets you want to book for Godzilla Minus Zero: 3
Booking successful!
Booking Details:
Movie: Godzilla Minus Zero
Tickets: 3
Total Price: Rs.540
```

Here the unit price is 180 and three tickets are booked, so the program shows $3 \times 180 = 540$.

6.4 Scenario 3: Viewing Bookings

```
Enter your choice: 3
-----Your Bookings-----
Booking 1
Movie: Godzilla Minus Zero
Tickets: 3
Total Price: Rs. 540
```

6.5 Scenario 4: Invalid Movie Number

```
Enter your choice: 2
...
Enter the number of the movie you want to book: 9
Invalid choice.
```

6.6 Scenario 5: Exiting

Enter your choice: 4

Exiting the program. Thank you for using the Movie Reservation System!

6.7 Worked Price Calculations

Movie	Unit Price	Tickets	Total
Ramayana	200	2	400
Godzilla Minus Zero	180	3	540
Varanasi	200	4	800
Spirit	200	1	200
The Angry Birds Movie 3	180	5	900

Table 6.1: Example totals computed by the system

7. Testing

7.1 Testing Strategy

Testing was carried out manually by running the program and feeding it a range of inputs. Three levels were considered: unit-level checks of individual functions, integration checks of the interaction among modules, and system-level checks of complete user scenarios. Both valid inputs and boundary or invalid inputs were used.

7.2 Test Cases

ID	Input / Action	Expected Result	Observed
T1	Menu: 1	Movie list displayed	Pass
T2	Menu: 2, movie 1, 2 tickets	Booking OK, total 400	Pass
T3	Menu: 2, movie 2, 3 tickets	Booking OK, total 540	Pass
T4	Menu: 3 with no bookings	'No bookings found.'	Pass
T5	Menu: 3 after two bookings	Both bookings listed as 1 and 2	Pass
T6	Menu: 2, movie 9	'Invalid choice.'	Pass
T7	Menu: 7	'Invalid choice. Please try again.'	Pass
T8	Menu: 4	Farewell message, program ends	Pass
T9	Book same movie twice	Two separate booking records	Pass
T10	Menu: abc (text)	Should show error and re-prompt	Fail: ValueError crash
T11	Movie number: abc	Should show error and re-prompt	Fail: ValueError crash
T12	Tickets: 0	Should be rejected	Fail: zero-value booking stored
T13	Tickets: -2	Should be rejected	Fail: negative total stored
T14	Tickets: 2.5 or blank	Should show error	Fail: ValueError crash

Table 7.1: Test cases and results

7.3 Test Summary

Of the fourteen test cases, nine passed and five failed. All failures relate to **input validation**: the program assumes that the user always types a valid whole number. When this assumption is violated, the program either terminates with an unhandled exception or records

meaningless data. These findings are analysed in Chapter 8 and fixed in Chapter 9.

Category	Cases	Passed	Failed
Valid scenarios	T1-T9	9	0
Invalid or boundary input	T10-T14	0	5
Total	14	9	5

Table 7.2: Summary of test results

8. Code Review: Limitations and Issues Identified

A careful review of the source code revealed several issues. They do not prevent the basic flow from working, but a robust application should address them.

8.1 Missing Input Validation

Every numeric prompt uses `int(input(...))` directly. If the user types a letter, symbol, decimal number or just presses Enter, Python raises a `ValueError` and the program stops. This affects the main menu choice, the movie number and the ticket count.

8.2 Non-Positive Ticket Counts

The ticket count is never checked. A value of zero creates a free booking and a negative value produces a negative total price, which could be misused to reduce a sum in a larger system.

8.3 Duplicate `display_movies` Functions

The function `display_movies` is defined twice: once in *movies.py* without parameters and once in *booking.py* with a parameter. In *main.py*, the import comes from *movies.py*, so option 1 uses the first version, while the booking flow uses the second. Two copies of the same logic must be kept in sync and can confuse readers. A single shared function should be used.

8.4 Inconsistent Price Formatting

The movie list prints the price as a bare number (for example, *Ramayana* - 200) while booking confirmations use the rupee symbol. The list should show the currency consistently.

8.5 Data Is Not Persistent

Bookings live only in memory. Closing the program erases all of them. A real system must store records in a file or database.

8.6 Global State

The `bookings` list is a module-level global variable. This is acceptable in a small script but makes testing and extension harder. A class or a passed-in data store would be cleaner.

8.7 Currency Symbol and Encoding

The rupee sign is a non-ASCII character. Python 3 handles it well, but some older Windows consoles may fail to display it unless the console encoding is UTF-8. Using the text "Rs." or configuring the console avoids display problems.

8.8 Other Observations

- No confirmation step before finalising a booking.
- No ability to cancel or modify a booking.
- No show time, date, theatre or seat information.
- No cap on the number of tickets per booking.
- No running total across bookings.
- No automated tests are included in the project.

Issue	Severity	Fix Difficulty
Missing input validation	High	Easy
Zero/negative tickets	High	Easy
Duplicate display function	Medium	Easy
Inconsistent currency display	Low	Easy
No persistence	Medium	Medium
Global state	Low	Medium

Table 8.1: Summary of identified issues

9. Recommended Fixes and Improved Code

9.1 Safe Integer Input Helper

A small reusable helper repeatedly asks until the user provides a valid whole number, optionally within a range. It eliminates crashes caused by non-numeric input.

```
def read_int(prompt, minimum=None, maximum=None):
    while True:
        text = input(prompt).strip()
        try:
            value = int(text)
        except ValueError:
            print("Please enter a whole number.")
            continue
        if minimum is not None and value < minimum:
            print(f"Value must be at least {minimum}.")
        elif maximum is not None and value > maximum:
            print(f"Value must be at most {maximum}.")
        else:
            return value
```

9.2 Improved book_ticket()

```
MAX_TICKETS = 10

def book_ticket(movies):
    display_movies(movies)
    choice = read_int("Enter the number of the movie you want to book: ")
    if choice not in movies:
        print("Invalid choice.")
        return
    movie_name, price = movies[choice]
    tickets = read_int(f"Number of tickets for {movie_name} (1-{MAX_TICKETS}): ",
                      minimum=1, maximum=MAX_TICKETS)
    total = tickets * price
    bookings.append({"movie": movie_name, "tickets": tickets, "total": total})
    print("Booking successful!")
    print(f"Movie: {movie_name}")
    print(f"Tickets: {tickets}")
    print(f"Total Price: Rs.{total}")
```

9.3 Single display_movies Function

Keep only the parameterised version and delete the copy in *movies.py*. Update *main.py* to import from *booking.py* (or a new *utils.py*) and call `display_movies(movies)`.

```
def display_movies(movies):
    print("\n-----Available Movies-----")
    for number, (title, price) in movies.items():
        print(f"{number}. {title} - Rs.{price}")
```

9.4 Safe Main Loop

```
from movies import movies
from booking import book_ticket, display_bookings, display_movies, read_int
from display import show_menu
```

```
while True:
    show_menu()
    choice = read_int("Enter your choice: ")
    if choice == 1:
        display_movies(movies)
    elif choice == 2:
        book_ticket(movies)
    elif choice == 3:
        display_bookings()
    elif choice == 4:
        print("Exiting the program. Thank you for using the Movie Reservation System!")
        break
    else:
        print("Invalid choice. Please try again.")
```

9.5 Saving Bookings to a File

Persistence can be added with the standard `json` module. Bookings are loaded when the program starts and saved after every successful reservation.

```
import json, os
FILE = "bookings.json"

def load_bookings():
    if os.path.exists(FILE):
        with open(FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    return []

def save_bookings(bookings):
    with open(FILE, "w", encoding="utf-8") as f:
        json.dump(bookings, f, indent=2)
```

9.6 Expected Effect on Test Results

Test	Before	After fixes
T10 Menu: abc	Crash	Re-prompt
T11 Movie: abc	Crash	Re-prompt
T12 Tickets: 0	Accepted	Rejected
T13 Tickets: -2	Accepted	Rejected
T14 Tickets: 2.5	Crash	Re-prompt

Table 9.1: Effect of the recommended fixes

10. Future Enhancements

The current design is a solid foundation on which many features can be layered. The following enhancements are recommended in rough order of effort.

10.1 Short-Term Improvements

- Input validation and error handling as described in Chapter 9.
- Cancellation of a booking by its number.
- A grand total across all bookings in the current session.
- Storing movies and bookings in JSON or CSV files.
- Adding an administrator menu to add, edit and remove movies.

10.2 Medium-Term Improvements

- **Show timings and dates:** allow multiple shows per movie.
- **Seat selection:** represent the hall as a grid and mark seats as booked.
- **User accounts:** log in with a name and password so that bookings belong to a user.
- **Object-oriented redesign:** classes such as Movie, Booking and Theatre.
- **Discounts and taxes:** apply GST, coupon codes and student concessions.
- **Unit tests:** use unittest or pytest for automated regression checks.

10.3 Long-Term Improvements

- A graphical interface using Tkinter or a web interface using Flask or Django.
- A relational database such as SQLite or MySQL for reliable storage.
- Integration with an online payment gateway.
- E-ticket generation with a QR code and email delivery.
- A mobile front end that talks to a REST API.

10.4 Suggested Class Design

```
class Movie:
    def __init__(self, number, title, price):
        self.number, self.title, self.price = number, title, price

class Booking:
    def __init__(self, movie, tickets):
        self.movie = movie
        self.tickets = tickets
        self.total = movie.price * tickets

class Theatre:
    def __init__(self, movies):
```

```
self.movies = {m.number: m for m in movies}  
self.bookings = []
```

Moving to classes groups data with the behaviour that operates on it, removes the need for global variables and makes the program easier to extend and test.

11. Conclusion

The Movie Reservation System successfully demonstrates a complete, menu-driven application built with core Python features. It presents the available movies, lets the user reserve tickets, computes prices automatically and allows the session's bookings to be reviewed. The separation into four modules keeps each file short and focused, and the use of a dictionary for the catalogue and a list of dictionaries for bookings shows a natural and efficient way of modelling the problem.

Testing showed that the program behaves correctly for valid input and gracefully handles an out-of-range movie number. It also revealed a set of weaknesses in input validation, duplicated code and data persistence. Chapter 9 proposes concrete fixes for each weakness, and Chapter 10 outlines a path towards a full-featured reservation platform.

Building this project reinforced the value of modular design, careful user-input handling and systematic testing. The lessons apply equally to much larger software systems, and the project provides a strong base for further learning in Python and software engineering.

12. References

- Python Software Foundation. *The Python Tutorial* and *The Python Standard Library*. docs.python.org/3.
- Python Software Foundation. *Built-in Types: Dictionaries, Tuples and Lists*. docs.python.org/3/library/stdtypes.html.
- Python Software Foundation. *Errors and Exceptions*. docs.python.org/3/tutorial/errors.html.
- Python Software Foundation. *json - JSON encoder and decoder*. docs.python.org/3/library/json.html.
- Matthes, E. *Python Crash Course*, No Starch Press.
- Sweigart, A. *Automate the Boring Stuff with Python*, No Starch Press.
- Sommerville, I. *Software Engineering*, Pearson Education.
- van Rossum, G., Warsaw, B., Coghlan, N. *PEP 8 - Style Guide for Python Code*. peps.python.org/pep-0008.

Appendix A: Complete Source Code

A.1 main.py

```
from movies import movies, display_movies
from booking import book_ticket, display_bookings
from display import show_menu
while True:
    show_menu()
    choice = int(input("Enter your choice: "))
    if (choice == 1):
        display_movies()
    elif (choice == 2):
        book_ticket(movies)
    elif (choice == 3):
        display_bookings()
    elif (choice == 4):
        print("Exiting the program. Thank you for using the Movie Reservation System!")
        break
    else:
        print("Invalid choice. Please try again.")
```

A.2 movies.py

```
movies = {
    1: ("Ramayana", 200),
    2: ("Godzilla Minus Zero",180),
    3: ("Varanasi", 200),
    4: ("Spirit",200),
    5: ("The Angry Birds Movie 3",180)
}

def display_movies():
    print("\n-----Available Movies-----")
    for number, details in movies.items():
        print(f"{number}. {details[0]} - {details[1]}")
```

A.3 booking.py

```
bookings = []
def book_ticket(movies):
    display_movies(movies)

    choice = int(input("Enter the number of the movie you want to book: "))
    if choice in movies:
        movie_name, price = movies[choice]
        tickets = int(input(f"Enter the number of tickets you want to book for {movie_name}: "))
        total = tickets * price
        booking = {
            "movie": movie_name,
            "tickets": tickets,
            "total": total
        }
        bookings.append(booking)
        print("Booking successful!")
        print("Booking Details:")
        print(f"Movie: {movie_name}")
        print(f"Tickets: {tickets}")
        print(f"Total Price: Rs.{total}")
    else:
```

```
        print("Invalid choice.")
def display_bookings():
    print("-----Your Bookings-----")
    if len (bookings) == 0:
        print("No bookings found.")
    else:
        for i in range(len(bookings)):
            print("Booking", i + 1)
            print("Movie:", bookings[i]["movie"])
            print("Tickets:", bookings[i]["tickets"])
            print("Total Price: Rs.", bookings[i]["total"])
def display_movies(movies):
    print("\n-----Available Movies-----")
    for number, details in movies.items():
        print(f"{number}. {details[0]} - {details[1]}")
```

A.4 display.py

```
def show_menu():
    print("-----Movie Reservation System-----")
    print("1. Display Available Movies")
    print("2. Reserve a Movie Ticket")
    print("3. Display Your Bookings")
    print("4. Exit")
```

Note: the rupee symbol in the original files is shown as "Rs." here because the PDF's built-in font has no glyph for it.

Appendix B: Glossary

Term	Meaning
Module	A Python file containing definitions that can be imported by other files.
Dictionary	A collection of key-value pairs with fast lookup by key.
Tuple	An ordered, immutable sequence of values.
List	An ordered, mutable sequence of values.
f-string	A string literal prefixed with f that embeds expressions inside braces.
Unpacking	Assigning the items of a sequence to several variables at once.
ValueError	Exception raised when a value has the right type but an unsuitable content, such as <code>int('abc')</code> .
Entry point	The file executed first to start the program (<code>main.py</code>).
Persistence	Saving data to disk so that it survives after the program ends.
Separation of concerns	Design principle of splitting a program by responsibility.
CLI	Command-line interface: interaction through typed text.

Appendix C: Installation and Usage Guide

C.1 Installing Python

- Download Python 3 from python.org and run the installer.
- Tick the option to add Python to PATH on Windows.
- Verify the installation with `python --version`.

C.2 Preparing the Project

- Create a folder named `MovieReservation`.
- Save `main.py`, `movies.py`, `booking.py` and `display.py` in that folder.
- Open a terminal and change into the folder.

C.3 Running

```
cd MovieReservation
python main.py
```

C.4 Customising the Catalogue

To add a movie, open `movies.py` and insert a new entry using the next unused number, for example:

```
6: ("New Movie Title", 220),
```

To change a price, edit the second value in the corresponding tuple. No other file needs to be changed.

C.5 Troubleshooting

Problem	Cause	Solution
<code>ModuleNotFoundError</code>	Terminal is not in the project folder	<code>cd</code> into the folder containing the four files
<code>ValueError</code> on input	Non-numeric text was typed	Enter digits only, or apply the fixes in Chapter 9
Rupee sign shows as ?	Console encoding is not UTF-8	Use <code>chcp 65001</code> on Windows or print <code>Rs.</code> instead
Bookings vanish	Data is held in memory only	Add file persistence (Section 9.5)